VISVESVARAYA TECHNOLOGICAL UNIVERSITY

"JnanaSangama", Belgaum -590014, Karnataka.

LAB REPORT
on

Machine Learning (23CS6PCMAL)

Submitted by

Deepthi M (1BM23CS088)

in partial fulfillment for the award of the degree of

BACHELOR OF ENGINEERING
in
COMPUTER SCIENCE AND ENGINEERING

B.M.S. COLLEGE OF ENGINEERING
(Autonomous Institution under VTU)
BENGALURU-560019
Jan-2026 to June-2026

B.M.S. College of Engineering,

Bull Temple Road, Bangalore 560019

(Affiliated To Visvesvaraya Technological University, Belgaum)

Department of Computer Science and Engineering

CERTIFICATE

This is to certify that the Lab work entitled "Machine Learning (23CS6PCMAL)" carried out by **Deepthi M (1BM23CS088),** who is bonafide student of **B.M.S. College of Engineering.** It is in partial fulfillment for the award of **Bachelor of Engineering in Computer Science and Engineering** of the Visvesvaraya Technological University, Belgaum. The Lab report has been approved as it satisfies the academic requirements in respect of a Machine Learning (23CS6PCMAL) work prescribed for the said degree.

Saritha A.N Assistant Professor Department of CSE, BMSCE	Dr. Kavitha Sooda Professor & HOD Department of CSE, BMSCE

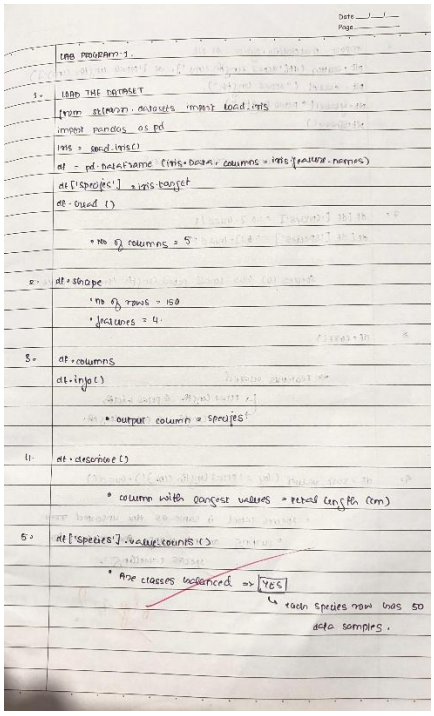
Index

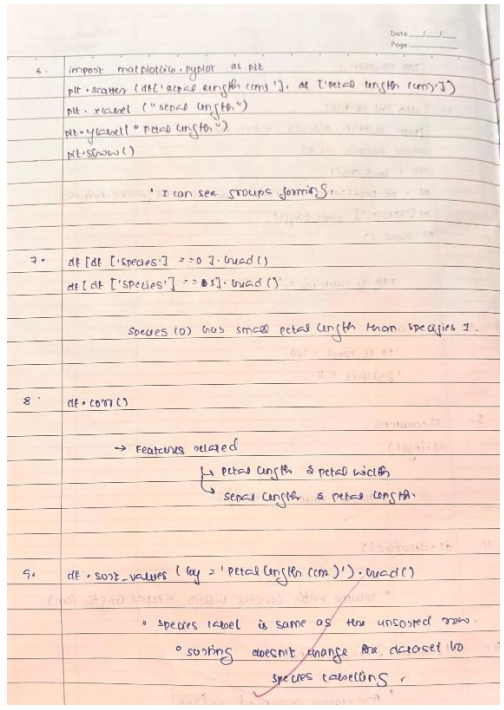
Sl. No.	Date	Experiment Title	Page No.
1	04-02-2026	Write a python program to import and export data using Pandas library functions	1
2	11-02-2026	Demonstrate various data pre-processing techniques for a given dataset	2-3
3	25-03-2026	Implement Linear and Multi-Linear Regression algorithm using appropriate dataset	4-5
4	25-03-2026	Build Logistic Regression Model for a given dataset	6-7
5	11-03-202	Use an appropriate data set for building the decision tree (ID3) and apply this knowledge to classify a new sample.	8-11
6	25-02-2026	Build KNN Classification model for a given dataset.	12-14
7	08-04-2026	Build Support vector machine model for a given dataset	15-16
8	08-04-2026	Implement Random forest ensemble method on a given dataset.	17-18
9	01-04-2026	Implement Boosting ensemble method on a given dataset.	19-20
10	01-04-2026	Build k-Means algorithm to cluster a set of data stored in a .CSV file.	21-22
11	08-04-2026	Implement Dimensionality reduction using Principal Component Analysis (PCA) method.	23-25

Github Link: https://github.com/Deepthi-1534/MACHINE-LEARNING-LAB

Program 1

Write a python program to import and export data using Pandas library functions

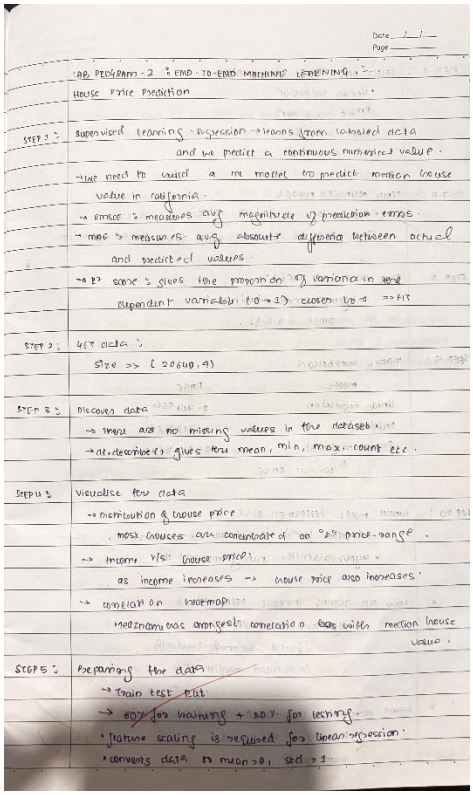


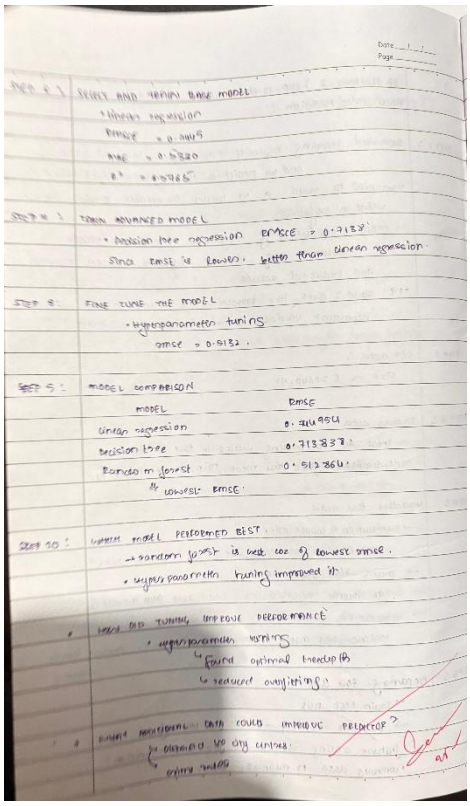



```
import pandas as pd
url = "https://raw.githubusercontent.com/mwaskom/seaborn-data/master/iris.csv"
df = pd.read_csv(url)
print(df.head())
df.to_csv("output.csv", index=False)
df.to_excel("output.xlsx", index=False)
df.to_json("output.json", orient="records")
df.to_html("output.html", index=False)
```

	sepal_length	sepal_width	petal_length	petal_width	species
0	5.1	3.5	1.4	0.2	setosa
1	4.9	3.0	1.4	0.2	setosa
2	4.7	3.2	1.3	0.2	setosa
3	4.6	3.1	1.5	0.2	setosa
4	5.0	3.6	1.4	0.2	setosa

Program – 2: Demonstrate various data pre-processing techniques for a given dataset

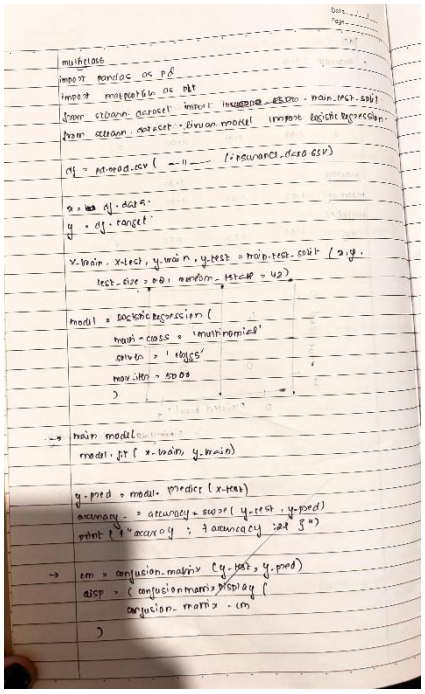


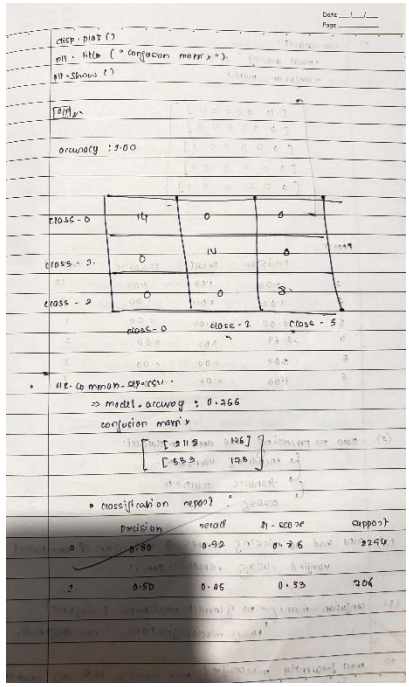


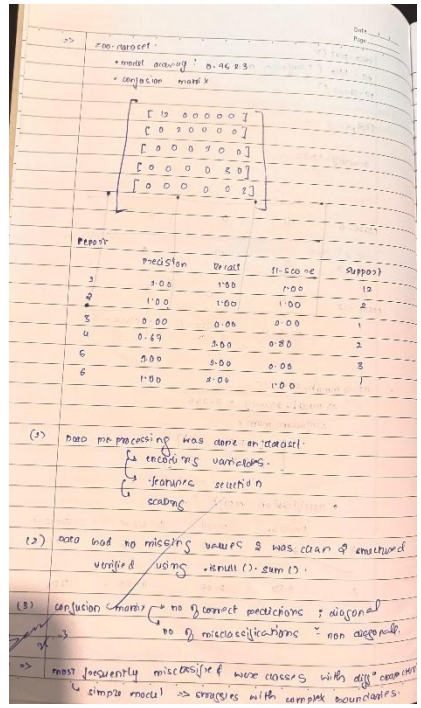

```
import numpy as np
import pandas as pd

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, MinMaxScaler, LabelEncoder

url = "https://raw.githubusercontent.com/mwaskom/seaborn-data/master/titanic.csv"
df = pd.read_csv(url)

print("Original Data:")
print(df.head())

df = df[['survived','pclass','sex','age','sibsp','parch','fare','embarked']]

df['age'].fillna(df['age'].median(), inplace=True)
df['embarked'].fillna(df['embarked'].mode()[0], inplace=True)

df.drop_duplicates(inplace=True)

le = LabelEncoder()
df['sex'] = le.fit_transform(df['sex'])
```

```
df['embarked'] = le.fit_transform(df['embarked'])

X = df.drop('survived', axis=1)
y = df['survived']

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

scaler_standard = StandardScaler()
X_train_std = scaler_standard.fit_transform(X_train)
X_test_std = scaler_standard.transform(X_test)

scaler_minmax = MinMaxScaler()
X_train_mm = scaler_minmax.fit_transform(X_train)
X_test_mm = scaler_minmax.transform(X_test)

print("\nAfter Preprocessing:")
print("Shape:", X_train.shape)
print("Standardized Sample:\n", X_train_std[:5])
print("MinMax Scaled Sample:\n", X_train_mm[:5])
```

Program-3 : Implement Linear and Multi-Linear Regression algorithm using appropriate dataset

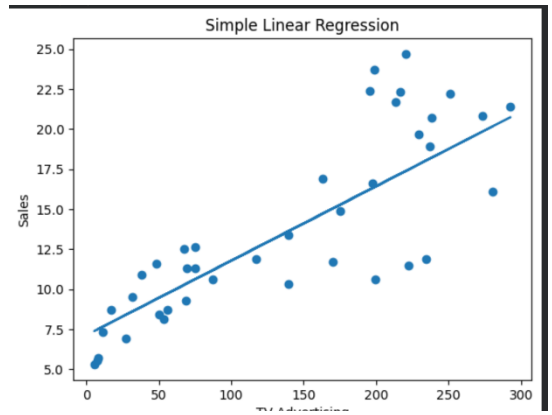






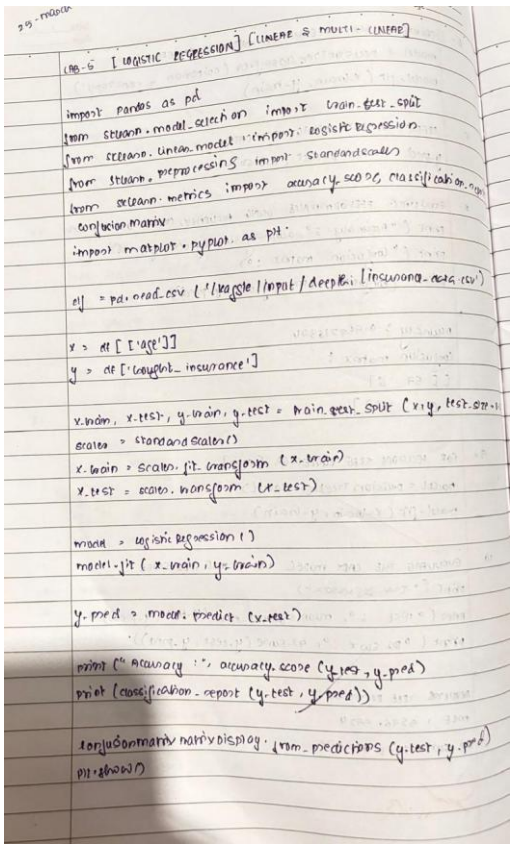
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score

url = "https://raw.githubusercontent.com/selva86/datasets/master/Advertising.csv"
df = pd.read_csv(url)

X = df[['TV']]
y = df['sales']

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

model = LinearRegression()
model.fit(X_train, y_train)

y_pred = model.predict(X_test)
```

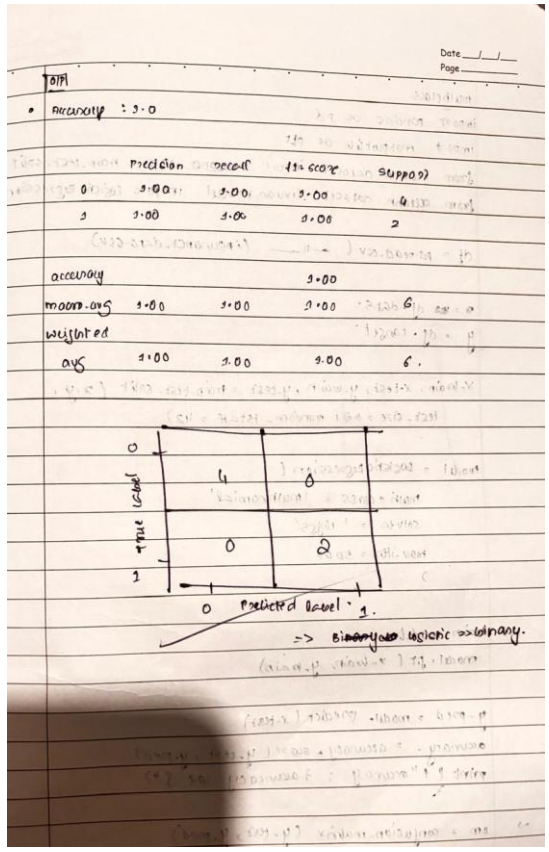
```
print(mean_squared_error(y_test, y_pred))
print(r2_score(y_test, y_pred))

plt.scatter(X_test, y_test)
plt.plot(X_test, y_pred)
plt.xlabel("TV Advertising")
plt.ylabel("Sales")
plt.title("Simple Linear Regression")
plt.show()
```




```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score

url = "https://raw.githubusercontent.com/selva86/datasets/master/Advertising.csv"
df = pd.read_csv(url)

X = df[['TV', 'radio', 'newspaper']]
y = df['sales']

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

model = LinearRegression()
model.fit(X_train, y_train)

y_pred = model.predict(X_test)

print(mean_squared_error(y_test, y_pred))
print(r2_score(y_test, y_pred))
3.1740973539761033
0.899438024100912
```

Program – 4 : Build Logistic Regression Model for a given dataset



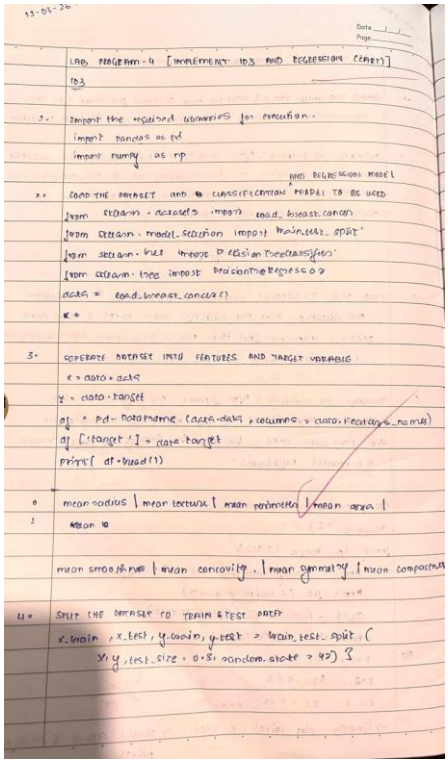


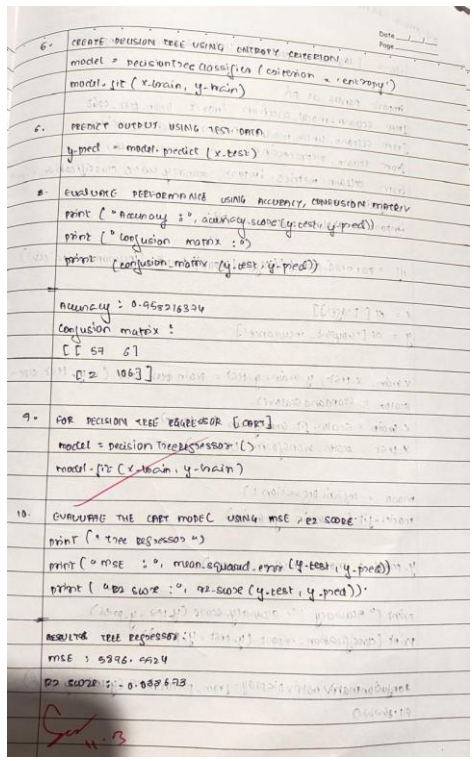

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

url = "https://raw.githubusercontent.com/jbrownlee/Datasets/master/pima-indians-diabetes.csv"

columns = [
    "pregnancies", "glucose", "blood_pressure", "skin_thickness",
    "insulin", "bmi", "diabetes_pedigree", "age", "outcome"
]

df = pd.read_csv(url, names=columns)
```

```
X = df.drop("outcome", axis=1)
y = df["outcome"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

model = LogisticRegression(max_iter=1000)
model.fit(X_train, y_train)

y_pred = model.predict(X_test)

print(accuracy_score(y_test, y_pred))
print(confusion_matrix(y_test, y_pred))
print(classification_report(y_test, y_pred))

y_prob = model.predict_proba(X_test)[:, 1]

plt.hist(y_prob, bins=10)
plt.xlabel("Predicted Probability")
plt.ylabel("Frequency")
plt.title("Prediction Probability Distribution")
plt.show()
```

```
Output : 0.7532467532467533
[[79 20]
 [18 37]]
              precision    recall  f1-score   support

           0       0.81      0.80      0.81        99
           1       0.65      0.67      0.66        55

    accuracy                           0.75       154
   macro avg       0.73      0.74      0.73       154
weighted avg       0.76      0.75      0.75       154
```

Program-5 : Use an appropriate data set for building the decision tree (ID3) and apply this knowledge to classify a new sample.





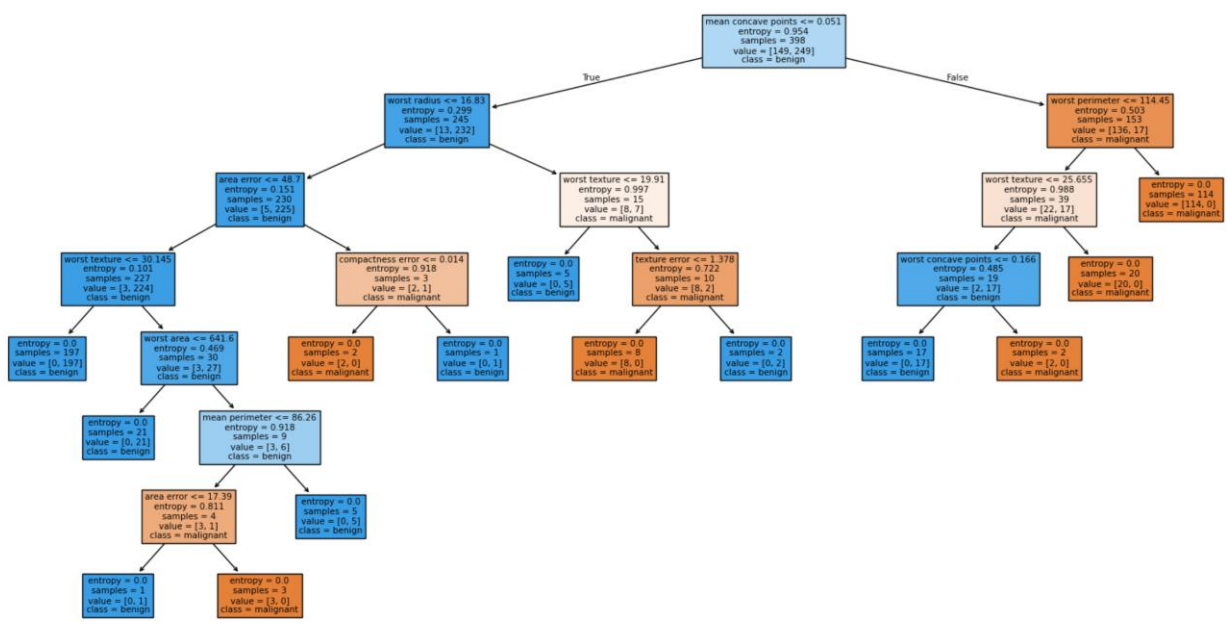
```
import pandas as pd
import numpy as np

from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier

from sklearn.metrics import accuracy_score
from sklearn.metrics import confusion_matrix
from sklearn.metrics import classification_report
data = load_breast_cancer()

X = data.data
y = data.target
df = pd.DataFrame(data.data, columns=data.feature_names)
df['target'] = data.target

print(df.head())
```

```
mean radius  mean texture  mean perimeter  mean area  mean smoothness  \
0       17.99        10.38          122.80     1001.0         0.11840
1       20.57        17.77          132.90     1326.0         0.08474
2       19.69        21.25          130.00     1203.0         0.10960
3       11.42        20.38           77.58      386.1         0.14250
4       20.29        14.34          135.10     1297.0         0.10030

   mean compactness  mean concavity  mean concave points  mean symmetry  \
0           0.27760          0.3001              0.14710         0.2419
1           0.07864          0.0869              0.07017         0.1812
2           0.15990          0.1974              0.12790         0.2069
3           0.28390          0.2414              0.10520         0.2597
4           0.13280          0.1980              0.10430         0.1809

   mean fractal dimension  ...  worst texture  worst perimeter  worst area  \
0                 0.07871  ...          17.33           184.60      2019.0
1                 0.05667  ...          23.41           158.80      1956.0
2                 0.05999  ...          25.53           152.50      1709.0
3                 0.09744  ...          26.50            98.87       567.7
4                 0.05883  ...          16.67           152.20      1575.0
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42
)
model = DecisionTreeClassifier(criterion='entropy')

model.fit(X_train, y_train)
y_pred = model.predict(X_test)
print("Accuracy:", accuracy_score(y_test, y_pred))
Accuracy: 0.9532163742690059
print("Confusion Matrix:")
print(confusion_matrix(y_test, y_pred))

Confusion Matrix:
[[ 57   6]
 [  2 106]]

print("Classification Report:")
print(classification_report(y_test, y_pred))
Classification Report:
```

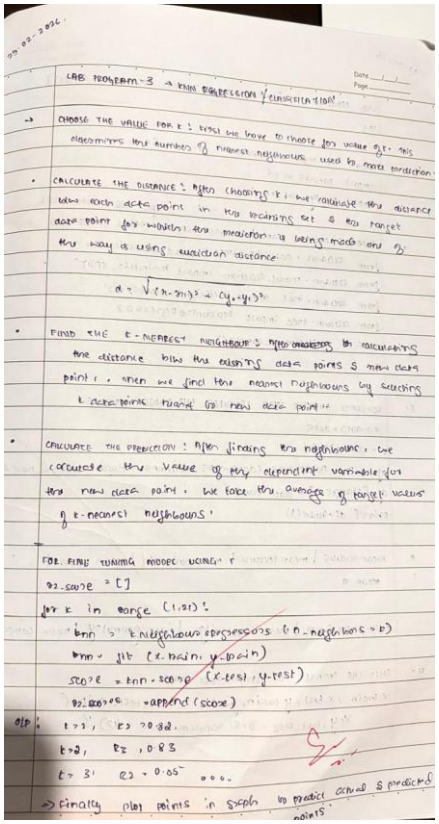
	precision	recall	f1-score	support
0	0.97	0.90	0.93	63
1	0.95	0.98	0.96	108
accuracy			0.95	171
macro avg	0.96	0.94	0.95	171
weighted avg	0.95	0.95	0.95	171

```
from sklearn.tree import plot_tree
import matplotlib.pyplot as plt

plt.figure(figsize=(20,10))

plot_tree(
    model,
    feature_names=data.feature_names,
    class_names=data.target_names,
    filled=True
)

plt.show()
```



```
import pandas as pd
import numpy as np

from sklearn.datasets import load_diabetes
from sklearn.datasets import fetch_california_housing

from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeRegressor

from sklearn.metrics import mean_squared_error
from sklearn.metrics import r2_score
data = load_diabetes()
_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42
)
model = DecisionTreeRegressor()

model.fit(X_train, y_train)
y_pred = model.predict(X_test)
print("Accuracy:", accuracy_score(y_test, y_pred))

print("Confusion Matrix")
print(confusion_matrix(y_test, y_pred))

print("Classification Report")
print(classification_report(y_test, y_pred))
Accuracy: 0.8333333333333334
Confusion Matrix
[[19  0  0]
 [ 2 18  1]
 [ 2  4  8]]
Classification Report
              precision    recall  f1-score   support

           0       0.83      1.00      0.90        19
           1       0.82      0.86      0.84        21
           2       0.89      0.57      0.70        14

    accuracy                           0.83        54
   macro avg       0.84      0.81      0.81        54
weighted avg       0.84      0.83      0.82        54
```

Program -6 : Build KNN Classification model for a given dataset.



LAB PROGRAM-3 → KNN REGRESSION / CLASSIFICATION

→ CHOOSE THE VALUE FOR K : First we have to choose for value of k. This determines the number of nearest neighbours used to make prediction.

• CALCULATE THE DISTANCE : After choosing k, we calculate the distance b/w each data point in the training set & the target data point for which the prediction is being made. One of the way is using euclidean distance.

$$d = \sqrt{(x_1-x_2)^2 + (y_1-y_2)^2}$$

• FIND THE K-NEAREST NEIGHBOUR : After calculating the distance b/w the existing data points & new data point, we find the nearest neighbours by selecting k data points nearest to new data point.

• CALCULATE THE PREDICTION : After finding the neighbours, we calculate the value of the dependent variable for the new data point. We take the average of target values of k-nearest neighbours.

FOR FINE TUNING MODEL USING K

```
R2_score = []
for k in range (1,21):
    knn = KNeighborsRegressor (n_neighbors = k)
    knn.fit (x_train, y_train)
    score = knn.score (x_test, y_test)
    R2_score.append (score)
```

o/p : k=1, R2 = 0.82.
k=2, R2 = 0.83
k=3, R2 = 0.05 ....

→ Finally plot points in graph to predict actual & predicted points.

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

url = "https://raw.githubusercontent.com/jbrownlee/Datasets/master/breast-cancer.csv"

columns = [
    "age", "menopause", "tumor_size", "inv_nodes", "node_caps",
    "deg_malig", "breast", "breast_quad", "irradiat", "class"
]

df = pd.read_csv(url, names=columns)

print("Dataset Preview:")
print(df.head())
```

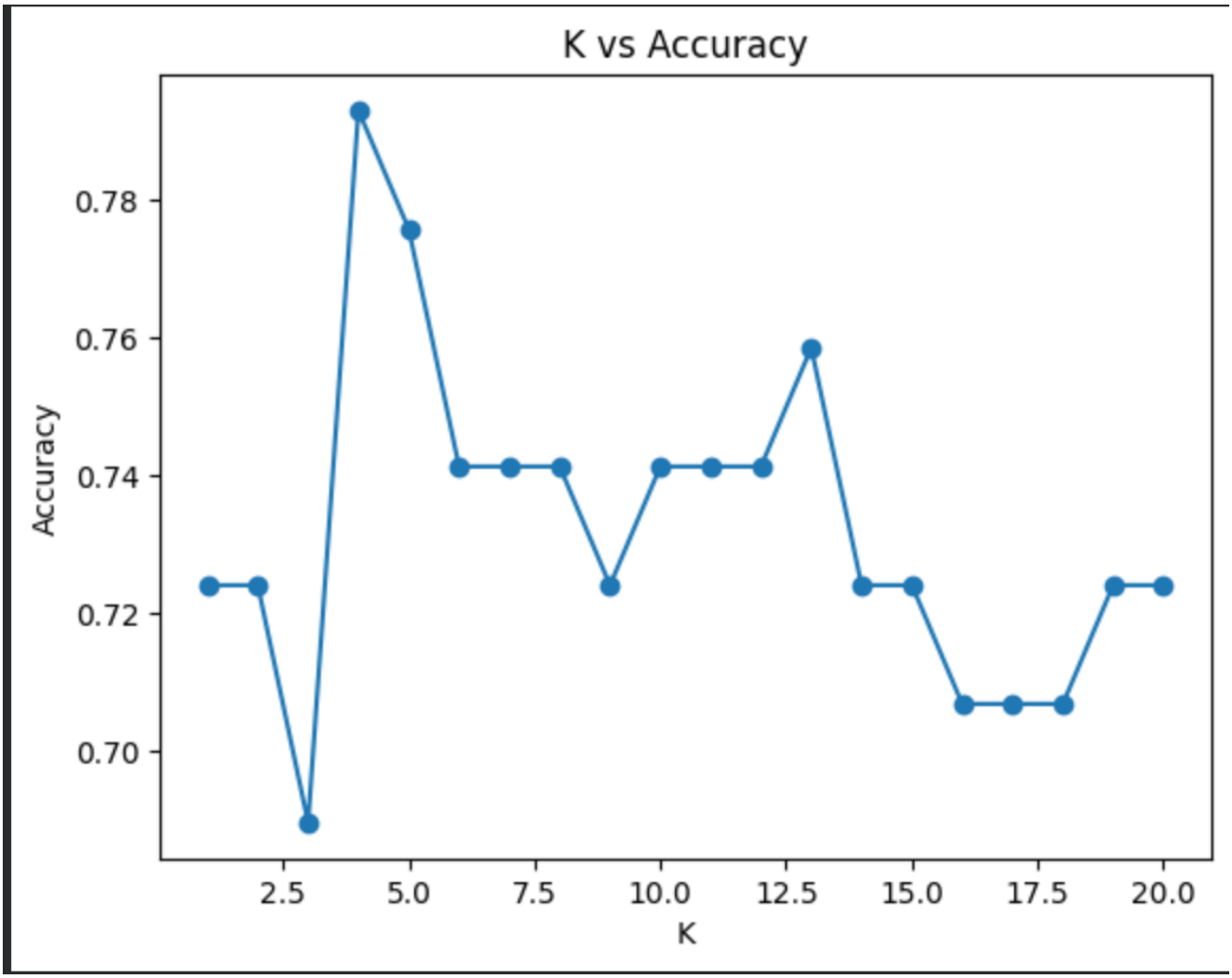
```
X = df.drop("class", axis=1)
y = df["class"]

X = pd.get_dummies(X)

# Train-test split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

model = KNeighborsClassifier(n_neighbors=5)
model.fit(X_train, y_train)

y_pred = model.predict(X_test)

print("\nAccuracy:", accuracy_score(y_test, y_pred))

print("\nConfusion Matrix:")
print(confusion_matrix(y_test, y_pred))

print("\nClassification Report:")
print(classification_report(y_test, y_pred))

accuracy_scores = []

for k in range(1, 21):
    knn = KNeighborsClassifier(n_neighbors=k)
    knn.fit(X_train, y_train)
    preds = knn.predict(X_test)
    accuracy_scores.append(accuracy_score(y_test, preds))

# Plot
plt.plot(range(1, 21), accuracy_scores, marker='o')
plt.xlabel("K")
plt.ylabel("Accuracy")
plt.title("K vs Accuracy")
plt.show()
```

Accuracy: 0.7758620689655172

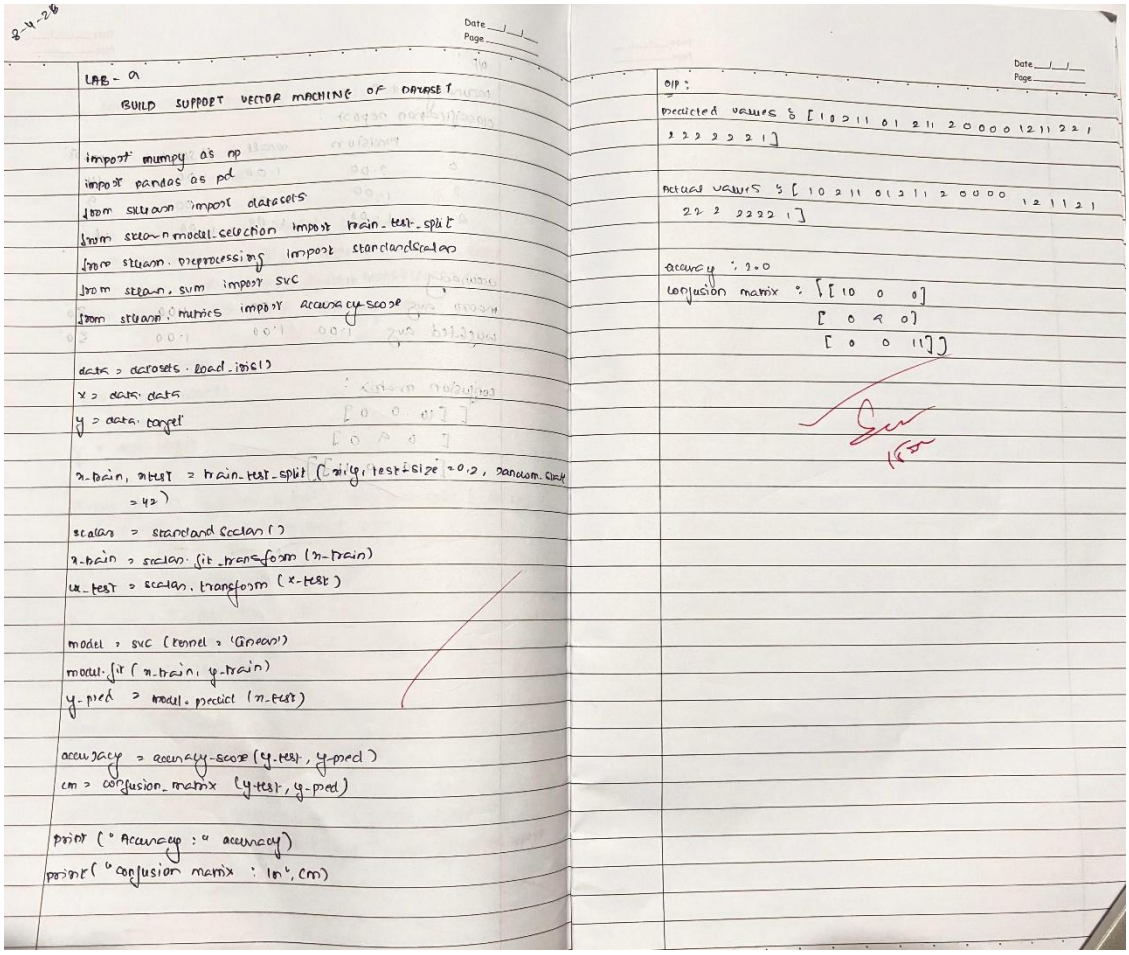
Confusion Matrix:
[[39 1]
 [12 6]]

Classification Report:

	precision	recall	f1-score	support
'no-recurrence-events'	0.76	0.97	0.86	40
'recurrence-events'	0.86	0.33	0.48	18
accuracy			0.78	58
macro avg	0.81	0.65	0.67	58
weighted avg	0.79	0.78	0.74	58



Program-7 : Build Support vector machine model for a given dataset



LAB - 9

BUILD SUPPORT VECTOR MACHINE OF DATASET

```
import numpy as np
import pandas as pd
from sklearn import datasets
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import standardScalar
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score

data = datasets.load_iris()
X = data.data
y = data.target

X_train, X_test = train_test_split(X, y, test_size=0.2, random_state
=42)
scalar = standardScalar()
X_train = scalar.fit_transform(X_train)
X_test = scalar.transform(X_test)

model = SVC(kernel='linear')
model.fit(X_train, y_train)
y_pred = model.predict(X_test)

accuracy = accuracy_score(y_test, y_pred)
cm = confusion_matrix(y_test, y_pred)

print("Accuracy:", accuracy)
print("confusion matrix:\n", cm)
```

```
O/P :
predicted values : [1 0 2 1 1 0 1 2 1 1 2 0 0 0 0 1 2 1 1 2 2 1
2 2 2 2 2 2 1]

Actual values : [1 0 2 1 1 0 1 2 1 1 2 0 0 0 0 1 2 1 1 2 1
2 2 2 2 2 2 1]

accuracy : 1.0
confusion matrix : [[10 0 0]
                    [ 0 9 0]
                    [ 0 0 11]]
```

import numpy as np
import pandas as pd

from sklearn import datasets
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, confusion_matrix

# Load dataset
data = datasets.load_iris()

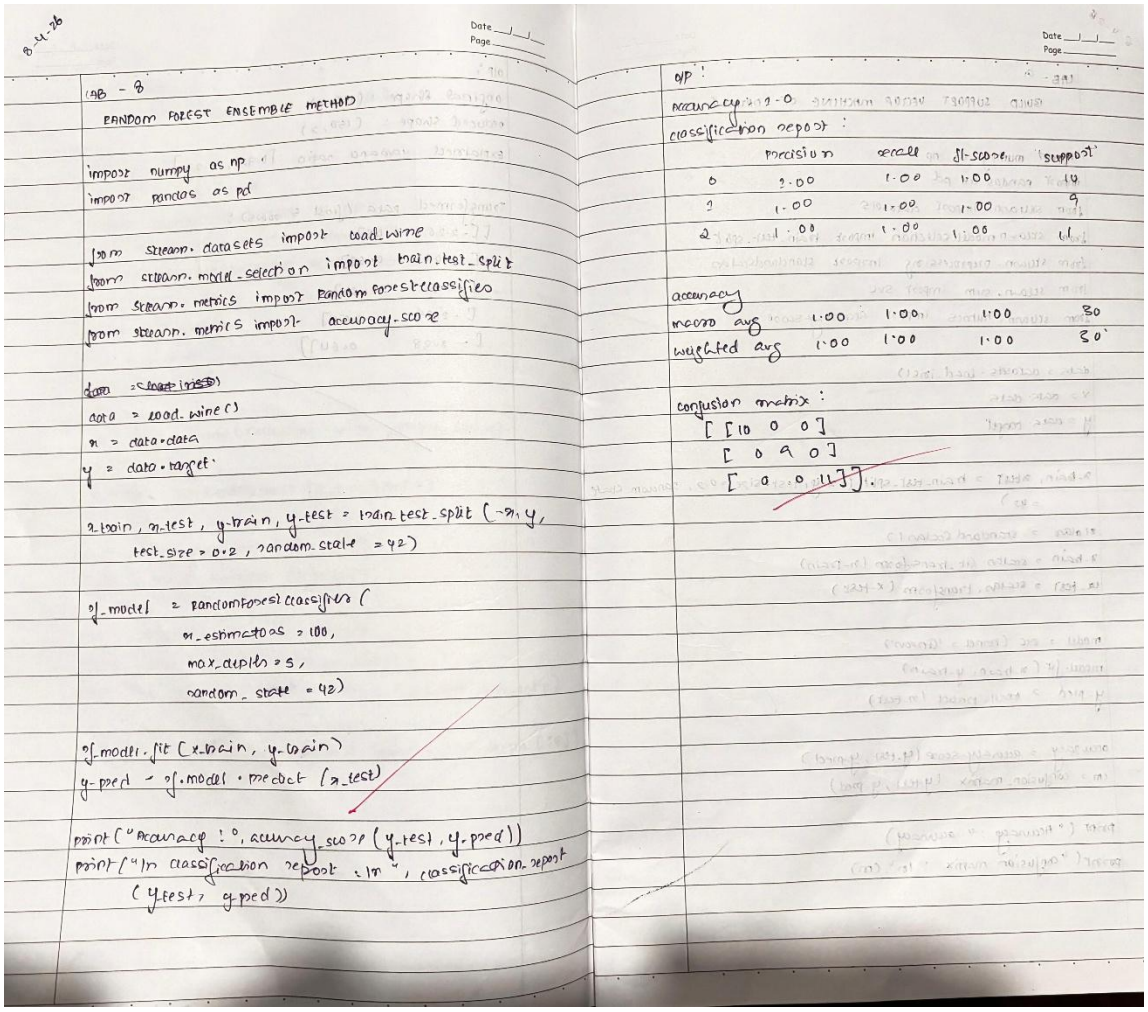
X = data.data
y = data.target

Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42

```
)

# Feature scaling
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# Model
model = SVC(kernel='linear')

# Train
model.fit(X_train, y_train)

# Predict
y_pred = model.predict(X_test)

# Evaluation
accuracy = accuracy_score(y_test, y_pred)
cm = confusion_matrix(y_test, y_pred)

print("Predicted values:", y_pred)
print("Actual values:", y_test)
print("Accuracy:", accuracy)
print("Confusion Matrix:\n", cm)
```

```
Predicted values: [1 0 2 1 1 0 1 2 2 1 2 0 0 0 0 1 2 1 1 2 0 2 0 2 2 2 2 2 0 0]
Actual values: [1 0 2 1 1 0 1 2 1 1 2 0 0 0 0 1 2 1 1 2 0 2 0 2 2 2 2 2 0 0]
Accuracy: 0.9666666666666667
Confusion Matrix:
 [[10  0  0]
 [ 0  8  1]
 [ 0  0 11]]
```

Program-8 : Implement Random forest ensemble method on a given dataset.

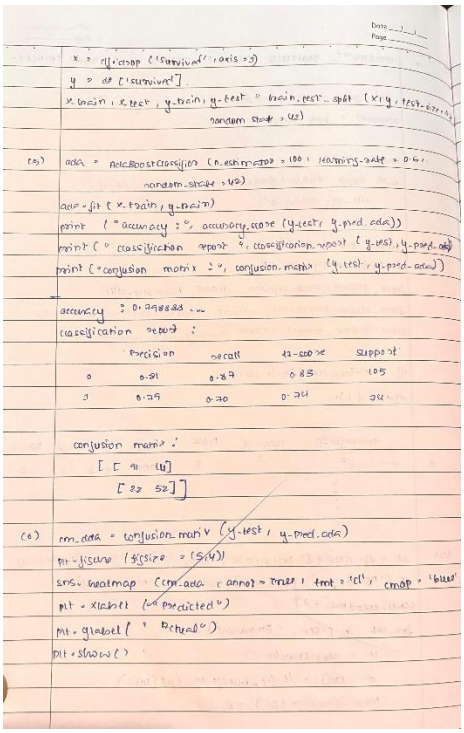


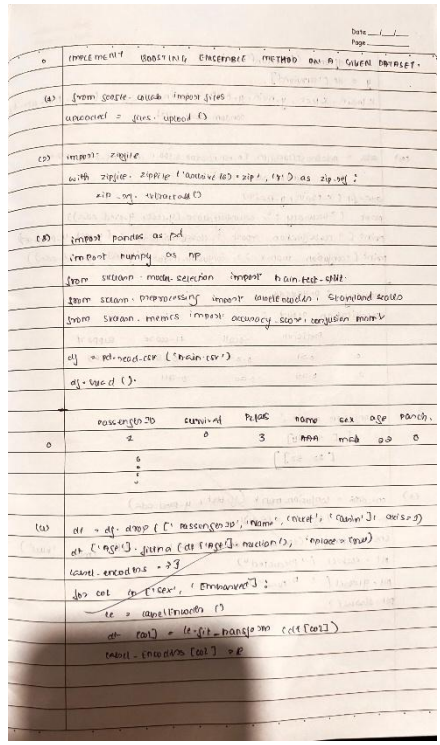
```
import numpy as np
import pandas as pd

from sklearn.datasets import load_wine
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix

# Load dataset
data = load_wine()

X = data.data
y = data.target

# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
```

```
    X, y, test_size=0.2, random_state=42
)

# Model
model = RandomForestClassifier(
    n_estimators=100,
    max_depth=5,
    random_state=42
)

# Train
model.fit(X_train, y_train)

# Predict
y_pred = model.predict(X_test)

# Evaluation
accuracy = accuracy_score(y_test, y_pred)
report = classification_report(y_test, y_pred)
cm = confusion_matrix(y_test, y_pred)

print("Accuracy:", accuracy)
print("\nClassification Report:\n", report)
```

```
•••  Accuracy: 1.0

     Classification Report:
                   precision    recall  f1-score   support

                0       1.00      1.00      1.00        14
                1       1.00      1.00      1.00        14
                2       1.00      1.00      1.00         8

            accuracy                           1.00        36
           macro avg       1.00      1.00      1.00        36
        weighted avg       1.00      1.00      1.00        36


     Confusion Matrix:
      [[14  0  0]
      [ 0 14  0]
      [ 0  0  8]]
```

Program-9 : Implement Boosting ensemble method on a given dataset.

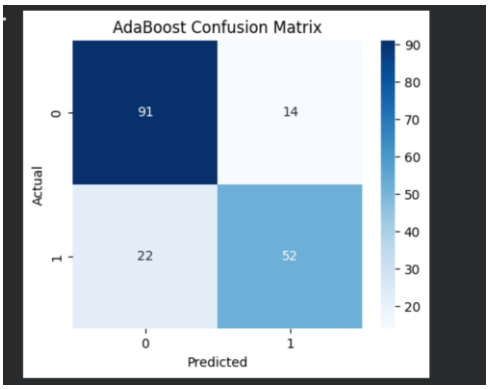




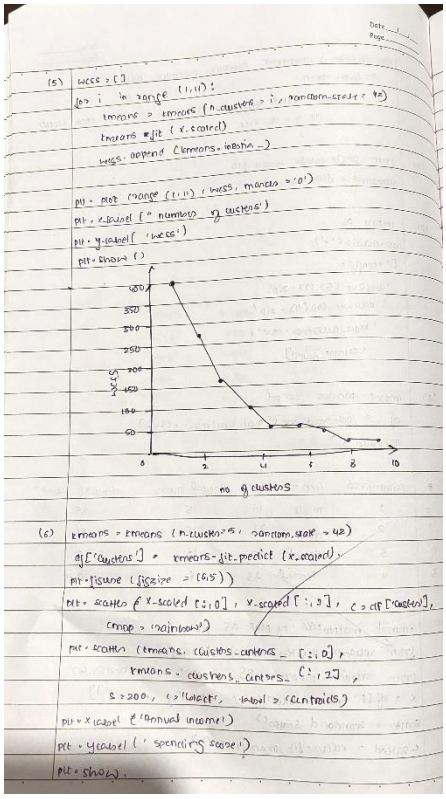
```
from google.colab import files
uploaded = files.upload()
import zipfile

# Extract ZIP to current directory
with zipfile.ZipFile('archive (6).zip', 'r') as zip_ref:
    zip_ref.extractall('.')
mport pandas as pd
import numpy as np

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.ensemble import AdaBoostClassifier, GradientBoostingClassifier
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix

# Step 2: Load dataset
df = pd.read_csv('train.csv')
df = df.drop(['PassengerId', 'Name', 'Ticket', 'Cabin'], axis=1)

# Fill missing values
df['Age'].fillna(df['Age'].median(), inplace=True)
df['Embarked'].fillna(df['Embarked'].mode()[0], inplace=True)
```

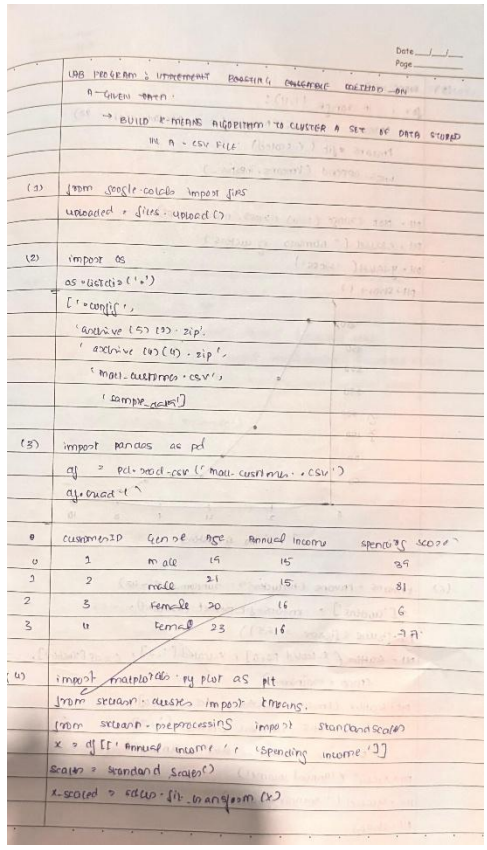
```
# Encode categorical variables
label_encoders = {}
for col in ['Sex', 'Embarked']:
    le = LabelEncoder()
    df[col] = le.fit_transform(df[col])
    label_encoders[col] = le

# Split features and target
X = df.drop('Survived', axis=1)
y = df['Survived']

# Train-test split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Scale features (optional but recommended for boosting)
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
ada = AdaBoostClassifier(n_estimators=100, learning_rate=0.5, random_state=42)
ada.fit(X_train_scaled, y_train)
y_pred_ada = ada.predict(X_test_scaled)

# Evaluate AdaBoost
print("AdaBoost Accuracy:", accuracy_score(y_test, y_pred_ada))
print("AdaBoost Classification Report:\n", classification_report(y_test, y_pred_ada))
print("AdaBoost Confusion Matrix:\n", confusion_matrix(y_test, y_pred_ada))
```

```
AdaBoost Accuracy: 0.7988826815642458
AdaBoost Classification Report:
               precision    recall  f1-score   support

           0       0.81      0.87      0.83       105
           1       0.79      0.70      0.74        74

    accuracy                           0.80       179
   macro avg       0.80      0.78      0.79       179
weighted avg       0.80      0.80      0.80       179

AdaBoost Confusion Matrix:
 [[91 14]
 [22 52]]
```



Program-10: Build k-Means algorithm to cluster a set of data stored in a .CSV file.





from google.colab import files

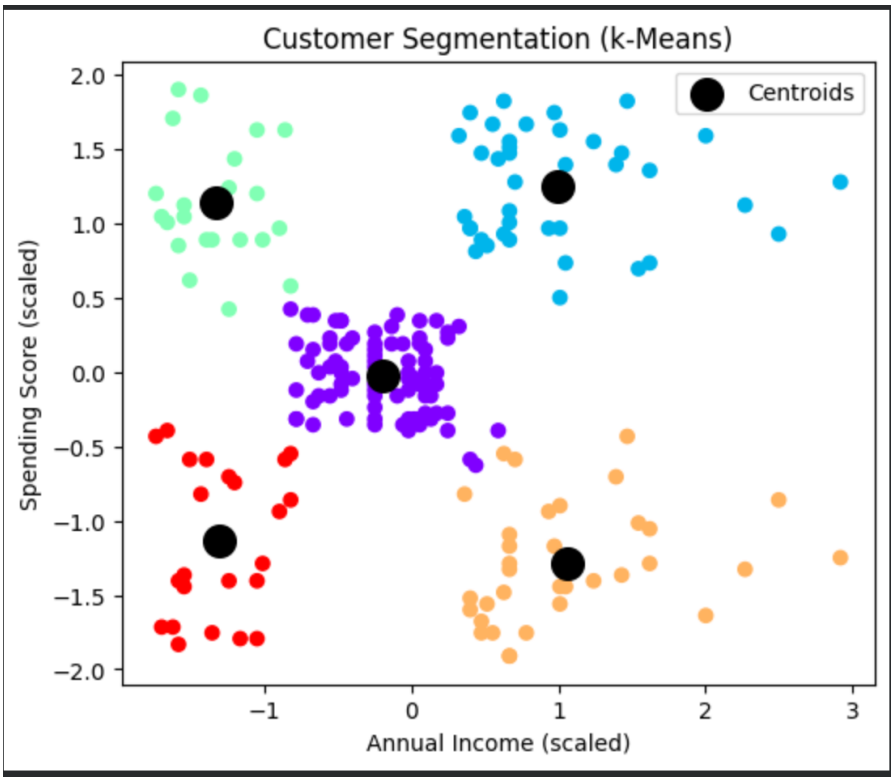
# Upload your ZIP file
uploaded = files.upload()
import pandas as pd

# Replace filename if your CSV has a different name
df = pd.read_csv('Mall_Customers.csv')

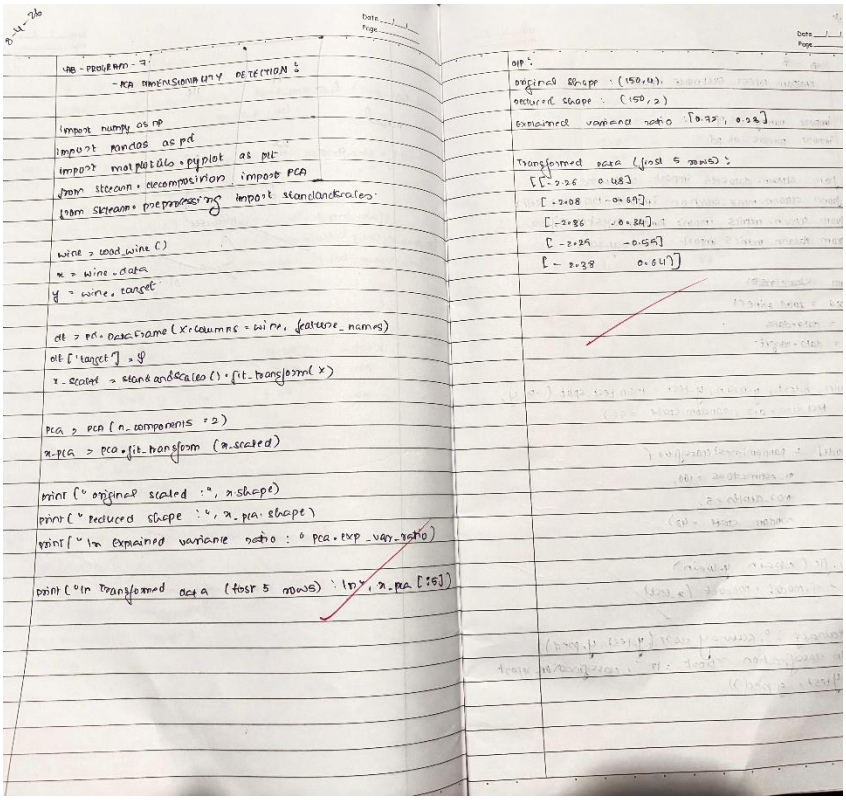
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler
X = df[['Annual Income (k$)', 'Spending Score (1-100)']]
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
wcss = []

for i in range(1, 11):
    kmeans = KMeans(n_clusters=i, random_state=42)
    kmeans.fit(X_scaled)
    wcss.append(kmeans.inertia_)

```
plt.plot(range(1, 11), wcss, marker='o')
plt.title('Elbow Method')
plt.xlabel('Number of clusters')
plt.ylabel('WCSS')
plt.show()

kmeans = KMeans(n_clusters=5, random_state=42)
df['Cluster'] = kmeans.fit_predict(X_scaled)
lt.figure(figsize=(6,5))
plt.scatter(X_scaled[:, 0], X_scaled[:, 1], c=df['Cluster'], cmap='rainbow')
plt.scatter(kmeans.cluster_centers_[:, 0],
            kmeans.cluster_centers_[:, 1],
            s=200, c='black', label='Centroids')
plt.xlabel('Annual Income (scaled)')
plt.ylabel('Spending Score (scaled)')
plt.title('Customer Segmentation (k-Means)')
plt.legend()
plt.show()
```



Program-11 : Implement Dimensionality reduction using Principal Component Analysis (PCA) method.

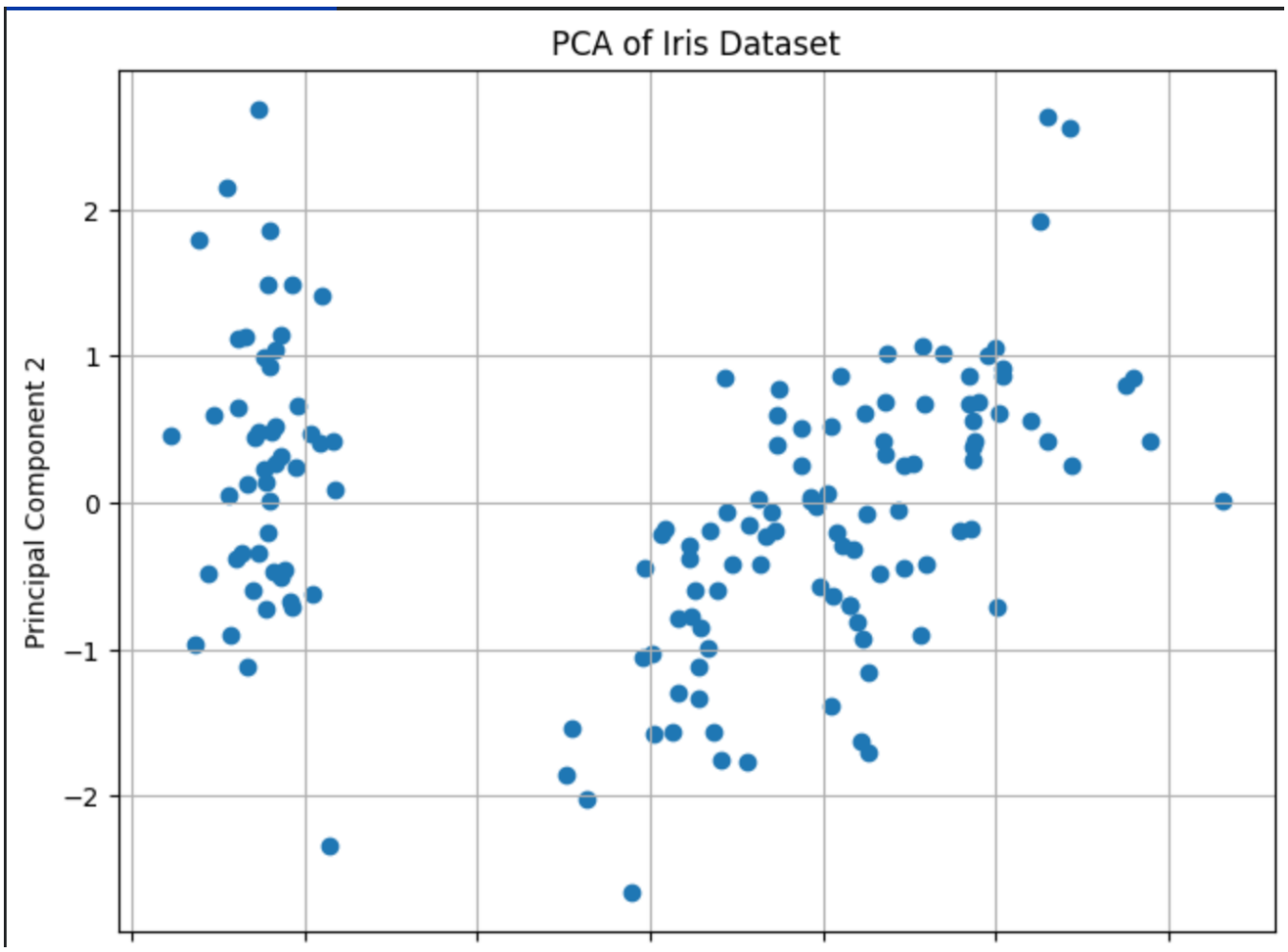


```
# Import libraries
import pandas as pd
import numpy as np
from sklearn.datasets import load_iris
from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler
import matplotlib.pyplot as plt

# Load built-in dataset
iris = load_iris()

# Convert to DataFrame
df = pd.DataFrame(iris.data, columns=iris.feature_names)

# Display first 5 rows
print("Original Dataset:")
print(df.head())

# Standardize the data
scaler = StandardScaler()
```

```
scaled_data = scaler.fit_transform(df)

# Apply PCA
pca = PCA(n_components=2)
principal_components = pca.fit_transform(scaled_data)

# Create new dataframe with PCA components
pca_df = pd.DataFrame(
    data=principal_components,
    columns=['Principal Component 1', 'Principal Component 2']
)

print("\nPCA Output:")
print(pca_df.head())

# Variance explained
print("\nExplained Variance Ratio:")
print(pca.explained_variance_ratio_)

# Plot graph
plt.figure(figsize=(8,6))
plt.scatter(
    pca_df['Principal Component 1'],
    pca_df['Principal Component 2']
)
plt.xlabel('Principal Component 1')
plt.ylabel('Principal Component 2')
plt.title('PCA of Iris Dataset')
plt.grid(True)
plt.show()
```

```
Original Dataset:
   sepal length (cm)  sepal width (cm)  petal length (cm)  petal width (cm)
0                5.1               3.5                1.4               0.2
1                4.9               3.0                1.4               0.2
2                4.7               3.2                1.3               0.2
3                4.6               3.1                1.5               0.2
4                5.0               3.6                1.4               0.2
```

PCA Output:

	Principal Component 1	Principal Component 2
0	-2.264703	0.480027
1	-2.080961	-0.674134
2	-2.364229	-0.341908
3	-2.299384	-0.597395
4	-2.389842	0.646835

Explained Variance Ratio:
[0.72962445 0.22850762]